UNIVERSIDAD AUTÓNOMA METROPOLITANA UNIDAD AZCAPOTZALCO

Division: Basic Sciences and Engineering

IMU Hardware Characterization, Firmware Modularization, and Full 6-Axis Data Acquisition on ESP32

Author: Zaldivar Peñaflor Eric Zuriel – 2183082002

Assignment: Microelectronics

Professor: JOSE LUIS ZAMORANO FLORES

Date: 12/07/2026

Índice

Abstract 4
1. Introduction 4
2.Hardware configuration 4
2.1 Microcontroller — ESP32 DOIT DevKit V1 4
2.2 Inertial Sensor Module 4
2.3 OLED Display (attempted) 4
3. Electrical Connections 4
3.1 IMU 4
3.2 I2C Bus Notes 5
4. Phase 1 — Initial Hardware Characterization 5
4.1 I2C Bus Validation 5
4.2 Device Identification 5
4.3 Power Management 5
4.4 Configuration Registers 5
4.5 Raw Accelerometer Data 5
5. Phase 2 — Firmware Modularization and Full 6-Axis Acquisition 5
5.1 Code Architecture 5
5.2 Gyroscope Data Acquisition 5
5.3 Scale Conversion 6
5.4 Offset Calibration 6
5.5 Calibrated Output — Stationary 6
6. OLED Display Integration Attempt 6
7. Diagnostic Tooling 6
8. Key Findings 7
9. Implications for TinyML Development 7
10. Future Work 7
Conclusion 7
11. Phase 3 — Complementary Filter and Orientation Estimation 7
11.1 Motivation 7
11.2 Algorithm 7
11.3 Implementation 8
11.4 Verified Output (stationary) 8
12. Updated Future Work 8
13. Phase 4 — Dataset Collector Firmware 8
13.1 Motivation 8
13.2 Architecture 8
13.3 Session-Based Recording Protocol 8
13.4 Windowing and Feature Extraction 9
13.5 Output Format 9

13.6 Issue Identified and Corrected Prior to Commit 9
13.7 Current Status 9
14. Build System Restructuring — Multi-Environment Source Isolation 9
14.1 Motivation 9
14.2 Root Cause of the Original Approach's Failure 9
14.3 Adopted Solution: build_src_filter 10
14.4 OLED Retest — Deferred, Environment Ready 10
14.5 Verification 10
15. Updated Future Work (Post Phase 4) 10
16. Phase 5 — TinyML Model Training and On-Device Inference 10
16.1 Dataset Capture Summary 10
16.2 Exploratory Data Analysis 11
16.3 Feature Normalization 12
16.4 Model Architecture and Training 12
16.5 Quantization and On-Device Deployment 12
16.6 Physical Feedback — LED Mapping 12
16.7 Current Status and Known Issues 13
17. Updated Future Work (Post Phase 5) 13

Abstract

This report documents the first two development phases of a TinyML-oriented embedded systems project based on an ESP32 DOIT DevKit V1 and a commercially labeled MPU-6050 IMU module.

Phase 1 established low-level I2C communication, confirmed device identity through direct register interrogation, and validated raw accelerometer data acquisition without vendor libraries. Unexpectedly, the sensor returned WHO_AM_I = 0x72 instead of the MPU-6050 standard value of 0x68, indicating a pin-compatible replacement device.

Phase 2 extended the firmware to include full 6-axis acquisition (accelerometer and gyroscope), offset calibration, scale conversion to physical units (m/s2 and deg/s), and a modular code architecture separating hardware logic from the main application loop. An OLED display integration attempt was conducted and documented as a hardware failure due to a defective unit.

1. Introduction

TinyML applications depend on reliable, calibrated sensor pipelines. Before collecting training datasets or developing inference models, the sensing subsystem must be fully validated at every layer: physical connections, communication protocol, register behavior, scale factors, and noise characteristics.

This report covers:

- I2C bus validation and device identification
- Accelerometer and gyroscope register characterization
- Offset calibration and scale conversion
- Firmware modularization
- OLED display integration attempt and diagnosis
- I2C diagnostic tooling

2.Hardware configuration

2.1 Microcontroller — ESP32 DOIT DevKit V1

Specification	Value
Architecture	Dual-core Xtensa LX6
Clock frequency	240 MHz
Wireless	Wi-Fi + Bluetooth
USB-UART bridge	CP2102

2.2 Inertial Sensor Module

Commercial label: MPU-6050 | Actual device: Unknown — WHO_AM_I = 0x72 | Protocol: I2C | Address: 0x68

Available pins: VCC, GND, SDA, SCL, XDA, XCL, ADO, INT

2.3 OLED Display (attempted)

Label: DST-015 | Expected controller: SSD1306 | Interface: I2C

Status: Defective — unit did not respond on I2C bus and caused bus collapse when connected. Excluded from current hardware configuration.

3. Electrical Connections

3.1 IMU

ESP32 Pin	IMU Pin
3V3	VCC
GND	GND
GPIO21	SDA
GPIO22	SCL

3.2 I2C Bus Notes

Pull-up resistors (10 kOhm) were added between 3V3 and SDA/SCL during OLED integration attempts. Removed after confirming the OLED was defective. The protoboard showed partial rail failures during testing; IMU placement was validated by I2C scan before each session.

4. Phase 1 — Initial Hardware Characterization

4.1 I2C Bus Validation

I2C address scan confirmed device presence at 0x68.

4.2 Device Identification

Register	Expected (MPU-6050)	Observed
WHO_AM_I (0x75)	0x68	0x72

The device is not a genuine MPU-6050. It is likely a pin-compatible InvenSense/TDK-family replacement. The register map behaves identically to the MPU-6050 specification.

4.3 Power Management

Register	Initial value	Action
0x6B	0x40 (sleep)	Write 0x00 to wake

Bidirectional register access confirmed.

4.4 Configuration Registers

Register	Value	Interpretation
ACCEL_CONFIG (0x1C)	0x00	+-2g range (default)
GYRO_CONFIG (0x1B)	0x00	+-250 deg/s range (default)

4.5 Raw Accelerometer Data

Registers 0x3B–0x40 read directly. Representative raw values (stationary):

Axis	Raw value
AX	-1876
AY	524
AZ	17556

5. Phase 2 — Firmware Modularization and Full 6-Axis Acquisition

5.1 Code Architecture

File	Role
src/main.cpp	Initialization and main loop
src/imu.cpp	IMU hardware logic
include/imu.h	Public interface declarations
tools/i2c_scan.cpp	Standalone I2C diagnostic tool

5.2 Gyroscope Data Acquisition

Gyroscope output registers 0x43–0x48 read and confirmed functional.

5.3 Scale Conversion

Sensor	Range	Scale factor
Accelerometer	+-2g	16384 LSB/g
Gyroscope	+-250 deg/s	131 LSB/deg/s

Accelerometer: (raw - offset) / 16384 x 9.81 = m/s2

Gyroscope: (raw - offset) / 131 = deg/s

5.4 Offset Calibration

200-sample stationary routine. AZ offset subtracts 1g (16384 LSB) to account for gravity.

Parameter	Value
Offset AX	-1411
Offset AY	467
Offset AZ	1261
Offset GX	185
Offset GY	355
Offset GZ	-103

5.5 Calibrated Output — Stationary

Accelerometer (m/s2):

Axis	Typical	Noise range
AX	~0.000	+-0.060
AY	~0.000	+-0.050
AZ	~9.810	+-0.080

Gyroscope (deg/s):

Axis	Typical	Noise range
GX	~0.000	+-0.350
GY	~0.000	+-0.350
GZ	~0.000	+-0.320

6. OLED Display Integration Attempt

A DST-015 OLED display module was connected to the I2C bus (GPIO21/GPIO22) to display real-time sensor data.

Findings:

- Unit did not appear in I2C address scans (0x3C and 0x3D absent).
- When connected, caused complete I2C bus collapse (Error -1 on all transactions).
- Failure persisted with 10 kOhm pull-up resistors on SDA and SCL.
- Failure persisted with VIN (5V) supply instead of 3V3.
- Connecting only VCC and GND (no SDA/SCL) still caused bus collapse — internal short circuit confirmed.

Conclusion: The OLED unit is defective. Integration deferred pending replacement hardware.

7. Diagnostic Tooling

A standalone I2C bus scanner was implemented as a separate PlatformIO environment ([env:i2c-scan]) with dedicated source directory (tools/i2c_scan.cpp). Allows bus diagnostics without modifying the main firmware.

8. Key Findings

- IMU is not a genuine MPU-6050 (WHO_AM_I = 0x72) but is fully register-compatible.
- ACCEL_CONFIG and GYRO_CONFIG default to 0x00: +-2g and +-250 deg/s confirmed.
- All 6 axes operational and calibrated.
- Calibrated accelerometer at rest: AX ~ AY ~ 0 m/s2, AZ ~ 9.81 m/s2.
- Calibrated gyroscope at rest: GX ~ GY ~ GZ ~ 0 deg/s.
- Protoboard has partially failed rails; IMU placement must be validated each session.
- OLED display unit is defective and excluded from current configuration.

9. Implications for TinyML Development

The sensing subsystem is fully operational for dataset collection. Calibrated 6-axis data at ~5 Hz provides the foundation for:

- Digital filtering (complementary filter, Kalman)
- Orientation estimation (roll, pitch)
- Windowed feature extraction
- Dataset labeling and collection
- TinyML model training and quantization
- On-device inference

10. Future Work

- Digital filtering implementation (complementary filter as next step)
- Roll/pitch estimation from fused accelerometer + gyroscope data
- OLED display integration with replacement hardware
- Dataset collection framework
- Noise characterization and spectral analysis

Conclusion

Phase 1 established hardware identity and low-level communication without vendor libraries. Phase 2 extended the system to full 6-axis calibrated acquisition with a modular firmware architecture. The sensing pipeline is validated, calibrated, and ready for the signal processing and dataset collection phases of the TinyML project.

11. Phase 3 — Complementary Filter and Orientation Estimation

11.1 Motivation

Raw gyroscope data accumulates drift over time when integrated to estimate angle. Raw accelerometer data is accurate in static conditions but highly sensitive to vibration and dynamic acceleration. A complementary filter addresses both limitations by fusing the two sources.

11.2 Algorithm

Orientation is estimated using the following fusion equation:

```
orientation = alpha * gyroscope_integration + (1 - alpha) *
accelerometer_angle
```

With alpha = 0.98, the gyroscope contributes 98% (smooth, low-latency response to motion) and the accelerometer contributes 2% (long-term drift correction). Roll and pitch are computed independently. Yaw estimation requires a magnetometer and is excluded.

Roll from accelerometer:

```
roll = atan2(ay, az) * 180 / pi
```

Pitch from accelerometer:

```
pitch = atan2(-ax, sqrt(ay^2 + az^2)) * 180 / pi
```

Gyroscope integration:

```
gyro_roll  = prev_roll  + gx * dt
gyro_pitch = prev_pitch + gy * dt
```

11.3 Implementation

File	Role
include/filter.h	Orientation struct, filter_init() and filter_update() interface
src/filter.cpp	Complementary filter implementation

dt is computed from millis() at each loop iteration for accurate time integration.

11.4 Verified Output (stationary)

Parameter	Value
Roll at rest	~0.00 deg
Pitch at rest	~0.00 deg
Noise range	+-0.35 deg

Orientation responds correctly to physical rotation of the sensor. No observable drift during short-duration tests.

12. Updated Future Work

- OLED display integration with replacement hardware
- Dataset collection framework
- Noise characterization and spectral analysis
- Feature extraction pipeline (windowed RMS, mean, FFT features)
- TinyML model training and quantization
- On-device inference and physical feedback (buzzer, LEDs)

13. Phase 4 — Dataset Collector Firmware

13.1 Motivation

Before training data can be assembled, the firmware must emit windowed, feature-extracted samples in the exact numerical format the model will later consume, rather than raw per-sample streams. Performing feature extraction on-device validates that the same extraction routine used for training is reusable at inference time, eliminating train/inference skew.

An initial design considered a physical push-button to mark individual gesture events. This was rejected: button-triggered capture is noisy and impractical for repeated single-hand data collection sessions. A session-based serial command protocol was adopted instead — one command starts a labeled recording session that persists across many consecutive gesture repetitions.

13.2 Architecture

File	Role
include/collector.h	Window size / sample rate constants, public interface
src/gesture-recognition/collector.cpp	Session state machine, feature extraction, CSV serialization

main.cpp was extended to call collector_init() once at startup and collector_check_command() / collector_add_sample() each loop iteration, after the IMU read and filter update. The complementary filter's orientation output (roll/pitch) is computed but intentionally not passed to the collector — consistent with the locked design constraint that gesture classification must be orientation-invariant.

13.3 Session-Based Recording Protocol

Command	Effect
s	Start/continue a SHAKE-labeled session (label 0)
r	Start/continue a ROTATE-labeled session (label 1)

i	Start/continue an IDLE-labeled session (label 2)
x	Stop the current session
h	Reprint the CSV header

A session remains active, silently emitting one CSV row per completed window, until x or a new label command is received.

13.4 Windowing and Feature Extraction

Parameter	Value
Sample rate	50 Hz (20 ms loop delay)
Window size	50 samples = 1 s
Features per window	12 (mean + RMS of ax, ay, az, gx, gy, gz)

Feature extraction runs entirely on-device per completed window, using the same calibrated accelerometer/gyroscope values produced by the Phase 2 pipeline. No absolute roll/pitch is included in the feature set.

13.5 Output Format

CSV emitted over serial, one row per completed window:

```
gesture,ax,ay,az,gx,gy,gz,ax_rms,ay_rms,az_rms,gx_rms,gy_rms,gz_rms
```

gesture is the numeric label captured at the start of the row's window (0 = shake, 1 = rotate, 2 = idle). Rows are assembled offline into per-class CSV files for the training pipeline.

13.6 Issue Identified and Corrected Prior to Commit

Code review prior to staging identified that _sampleCount was not reset on a label transition (s/r/i) or on x. In practice this allows a partially filled window from one session to carry over into the next, so a window can be assembled from samples belonging to two different gestures while being emitted under a single label — a silent mislabeling defect that would have degraded dataset quality from the first recording session onward. Fixed by resetting _sampleCount = 0 on every label transition and on stop, discarding any partial window at session boundaries.

A related build-configuration issue was corrected in platformio.ini: the i2c-scan and ssd1306-test tool environments declared their src_dir override in bare sections rather than inside their corresponding [env:...] blocks, where PlatformIO requires it. This was later superseded entirely by the Section 14 restructuring.

13.7 Current Status

Collector firmware compiles and runs correctly against the fixed session-boundary logic described above. No training data has been recorded yet — the next session begins the actual capture (≥200 windows per class for shake, rotate, idle).

14. Build System Restructuring — Multi-Environment Source Isolation

14.1 Motivation

As the project grew to include standalone diagnostic firmware (i2c_scan) alongside the main gesture-recognition application, and with OLED bring-up testing (ssd1306_test) planned as a distinct firmware, a mechanism was needed to build multiple independent setup()/loop() programs from a single repository without cross-contamination between builds.

14.2 Root Cause of the Original Approach's Failure

The initial approach attempted to override src_dir on a per-environment basis inside [env:NAME] blocks in platformio.ini. This is not supported by the PlatformIO Core version used by this project: src_dir is not a recognized per-environment option. Confirmed directly via build output:

```
Warning! Ignore unknown configuration option `src_dir` in section [env:i2c-scan]
```

With this misconfiguration, environments intended to build diagnostic tools were silently compiling the main firmware's src_dir instead — a defect that could go unnoticed, since the build itself did not fail, it simply compiled the wrong program.

14.3 Adopted Solution: build_src_filter

build_src_filter is supported per environment and allows include/exclude glob patterns within a single shared src_dir. The repository was restructured so that each independent firmware lives in its own subdirectory of src/, and each environment's build_src_filter includes only its own subdirectory while explicitly excluding the others:

Directory	Firmware	Environment
src/gesture-recognition/	Main application — IMU driver, complementary filter, dataset collector	gesture-recognition (default)
src/i2c_scan/	Standalone I2C bus diagnostic tool	i2c-scan
src/ssd1306_test/	OLED bring-up test (hardware currently defective, deferred)	ssd1306-test

```
build_src_filter = +<gesture-recognition/> -<i2c_scan/> -<ssd1306_test/>
```

default_envs = gesture-recognition in [platformio] ensures an unqualified pio run builds only the primary firmware. The previous tools/ directory, which held diagnostic tool source outside of src_dir and therefore could not be reached by build_src_filter, was removed; its contents were relocated into src/.

14.4 OLED Retest — Deferred, Environment Ready

The ssd1306-test environment and its Adafruit SSD1306/GFX library dependencies are now correctly isolated and ready to build in advance of the actual hardware retest. The underlying hardware issue documented in Section 6 remains unresolved and is not being re-investigated at this time — this restructuring only prepares the build environment so the retest can proceed later without further tooling work.

14.5 Verification

Each environment was confirmed to compile only its intended source file(s) via verbose build output (pio run -e <env> -v), checking the Compiling entries against the expected file set.

15. Updated Future Work (Post Phase 4)

- Confirm Python + TensorFlow/Keras environment
- Execute labeled recording sessions and assemble per-class CSV dataset
- Exploratory data analysis — verify class separability, decide on feature normalization/scaling
- Train Dense(64)→Dense(32)→Dense(3, softmax) baseline in Keras
- Post-training int8 quantization → TFLite → C header export
- On-device inference integration and tensor arena sizing
- OLED display integration with replacement hardware — ssd1306-test environment scaffolded and isolated, retest itself still deprioritized

16. Phase 5 — TinyML Model Training and On-Device Inference

16.1 Dataset Capture Summary

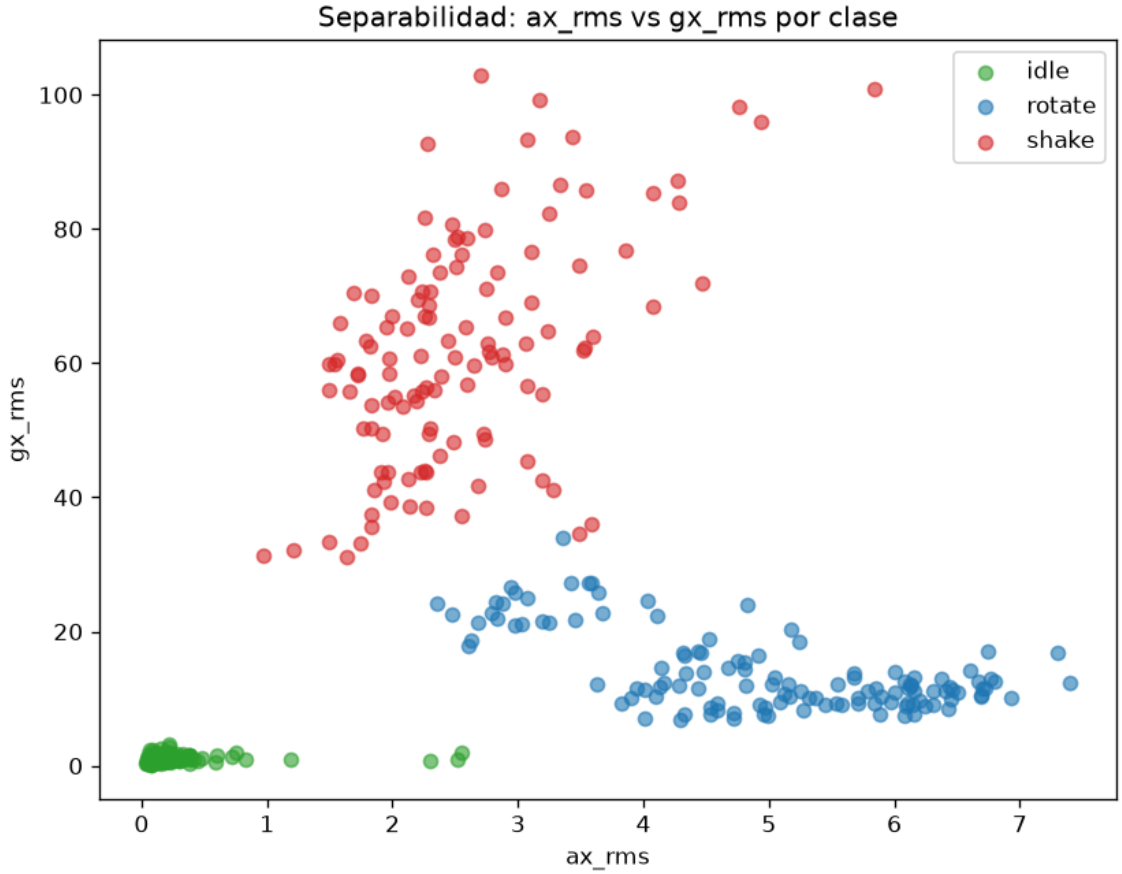
Labeled data was captured using the session-based collector protocol described in Section 13, across 5 distinct arm orientations per class (up, forward, down, diagonal-up, diagonal-down), 5 sessions of 10 seconds each. The first and last window of every session are discarded at capture time to remove the transition into and out of the gesture, per the protocol already documented.

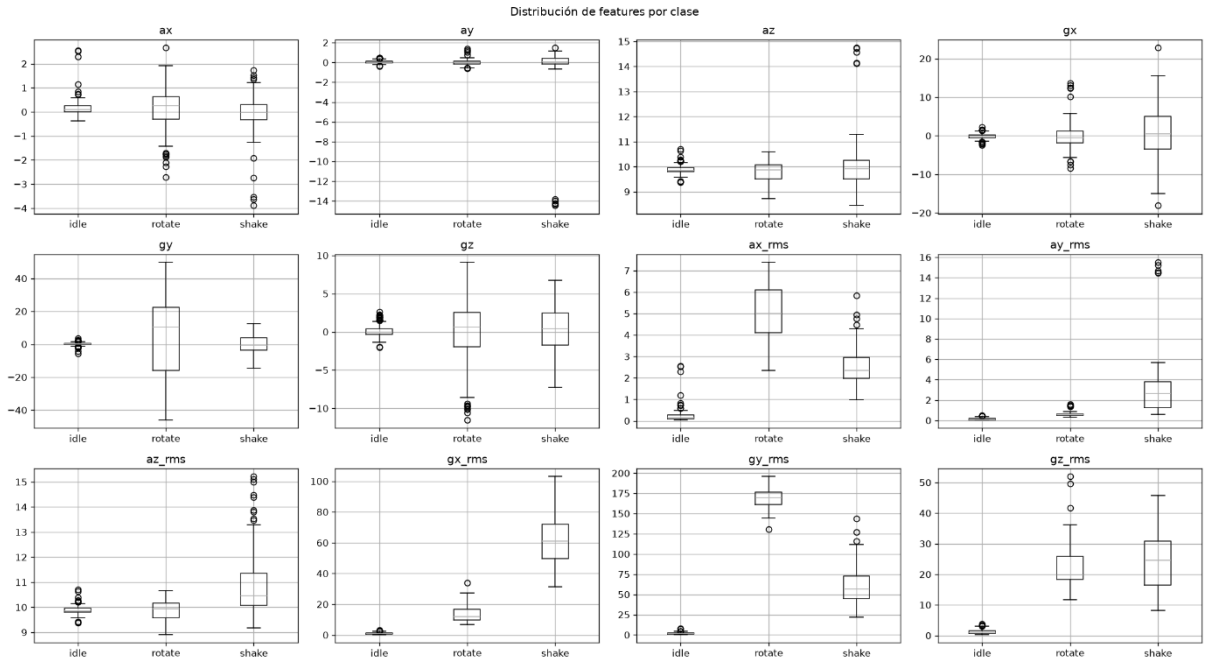
Class	Label	Windows captured
shake	0	120
rotate	1	125
idle	2	120
Total		365

Class balance is effectively even (within 5 windows across all three classes), so no resampling or class-weighting was required before training.

16.2 Exploratory Data Analysis

Per-class boxplots of all 12 features and a 2D scatter of the two most visually separating features (ax_rms vs gx_rms) were generated to verify separability before committing to a training run. The three classes form visually disjoint clusters with no observable overlap in the scatter projection. gy_rms is the single most discriminative feature (idle mean approx. 1.7, rotate approx. 168, shake approx. 61), consistent with rotate being a sustained rotation captured strongly on the gyroscope Y axis.





16.3 Feature Normalization

Features mix physical units (m/s2 for accelerometer, deg/s for gyroscope) with very different numeric ranges (e.g. gy_rms spans approximately 0.23 to 195, while ax spans approximately -3.9 to 2.7). StandardScaler (zero mean, unit variance per feature) was adopted to prevent gyroscope-derived features from dominating the training gradient purely due to magnitude. The scaler is fit only on the training split to avoid validation leakage, and its per-feature mean/standard deviation are exported for use on-device, since the same normalization must be applied to raw features before inference.

16.4 Model Architecture and Training

Layer	Output shape	Parameters
Dense (ReLU)	64	832
Dense (ReLU)	32	2,080
Dense (softmax)	3	99
Total		3,011

Training configuration: 80/20 stratified train/validation split, Adam optimizer, sparse categorical cross-entropy loss, 50 epochs, batch size 16.

Class	Precision	Recall	F1-score
shake	1.00	1.00	1.00
rotate	1.00	1.00	1.00
idle	1.00	1.00	1.00

Validation accuracy: 100% (73/73), with a fully diagonal confusion matrix (24 shake, 25 rotate, 24 idle, zero cross-class errors).

Methodological note (known limitation): the train/validation split was performed at the window level, not the session level. Because windows from the same 10-second session are highly autocorrelated, it is likely that windows from a given session appear in both the training and validation sets, which inflates the reported accuracy relative to a true out-of-session generalization measurement. Given the clean cluster separation already observed in the EDA scatter, true generalization performance is plausibly still high, but the exact 100% figure should not be read as an unbiased estimate. A session-level split is listed as a correction for a future iteration (Section 17).

16.5 Quantization and On-Device Deployment

The trained model was quantized post-training to full int8 (both input and output tensors) using the TFLite Converter, with a representative dataset built from the first 100 normalized training samples. Exported artifacts: gesture_model.tflite (8,200 bytes), model_data.h (C byte array for firmware inclusion), and scaler_params.h (per-feature mean and standard deviation for on-device normalization).

The initial choice of TFLite Micro runtime, espressif/esp-tflite-micro, failed to build under PlatformIO: its git distribution does not vendor its third-party dependencies (flatbuffers, gemmlowp/fixedpoint), which are normally resolved automatically by the ESP-IDF component manager but are invisible to PlatformIO's dependency resolution. This is a known limitation of consuming that repository outside of ESP-IDF. It was replaced with johnosbb/MicroTFLite, an Arduino/PlatformIO packaged distribution of the same runtime with all dependencies vendored, exposing a simplified API (ModelInit / ModelSetInput / ModelRunInference / ModelGetOutput) that also performs int8 quantization and dequantization internally.

Tensor arena: 8,192 bytes, a conservative starting allocation for a model of this size (3,011 parameters). A new firmware module, inference.h / inference.cpp, mirrors the window accumulation and mean/RMS feature extraction already implemented in collector.cpp, but runs continuously whenever collector_is_recording() is false — ensuring live inference and labeled dataset capture never operate on the same sample stream simultaneously.

16.6 Physical Feedback — LED Mapping

GPIO	Class	Behavior
25	shake	300 ms pulse, non-blocking
26	rotate	300 ms pulse, non-blocking
33	(reserved for flick_up — not yet implemented)	—

LED illumination is implemented as a stored turn-off timestamp checked once per loop iteration, rather than a blocking delay() call. The main loop's 20 ms delay is load-bearing for the IMU sample rate; a blocking delay on LED output would perturb the sampling window and was avoided for that reason.

[PHOTO: ESP32 + IMU + LED assembly on breadboard]

16.7 Current Status and Known Issues

- Live inference and dataset capture coexist correctly in the same firmware build, gated by collector_is_recording().
- Under normal, non-gesture arm movement, the classifier occasionally reports shake instead of idle. Root cause: the idle class was captured only as static, stationary poses across 5 orientations, and was never shown ambient low-level motion during training — a category gap in the dataset, not a firmware defect.
- Physical LED output was not yet confirmed functional at time of writing; hardware-level troubleshooting (polarity, isolated-pin test, breadboard continuity) is in progress.

17. Updated Future Work (Post Phase 5)

- Re-run the train/validation split at the session level, not the window level, to obtain an unbiased generalization estimate.
- Expand the idle class to include ambient, non-gesture motion, to reduce shake false positives during normal arm use.
- Complete LED hardware verification and wire GPIO33 once flick_up rule-based logic is implemented.
- Evaluate a fourth gesture class for the directional strike/stretch motion identified during capture — physically distinct from shake (single-direction impulse vs. sustained oscillation) and deliberately excluded from the current 3-class model.
- OLED display integration with replacement hardware (carried over from Section 14, still deprioritized).
- Commit Phase 5 deliverables to version control: inference.h/.cpp, model_data.h, scaler_params.h, the ml/ directory (capture_dataset.py, eda.py, train_model.py), and the updated main.cpp / platformio.ini.